

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
4ta práctica (tipo b)
Segundo Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales, NI OBJETOS** (con excepción de los elementos de `iostream`, `omanip` y `fstream`). **NO PUEDE UTILIZAR LA CLASE string**. Tampoco se podrán emplear las funciones `malloc`, `realloc`, `memset`, `strtok` o `strdup`, igualmente no se puede emplear cualquier función contenida en las bibliotecas `stdio.h`, `cstdio` o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ESTRICTO DISEÑO DESCENDENTE. **Cada función NO debe sobrepasar las 20 líneas de código aproximadamente**. El archivo `main.cpp` solo podrá contener la función `main` de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo `main.cpp` deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final**.
- El código comentado NO SE CALIFICARÁ. De igual manera NO SE CALIFICARÁ el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.

SE LES RECUERDA QUE, DE ACUERDO AL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- **Puntaje total:** 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLion siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"Lab04_2026_2_CO_PA_PN"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará los proyectos solicitados en la prueba.**
- En cada proyecto que desarrolle deberá crear una carpeta denominada **"Bibliotecas"** y allí coloque las bibliotecas de funciones que necesite. Dentro de la carpeta **"cmake-build-debug"** cree las carpetas **"ArchivosDeDatos"** y **"ArchivosDeReporte"** y allí respectivamente coloque el archivo de datos que se le proporcionará y el archivo del reporte solicitado en la prueba. De no colocar esto se descontará un punto en cada proyecto.

NO PODRÁ EMPLEAR ARREGLOS ESTÁTICOS DE MÁS DE UNA DIMENSIÓN

NO PUEDE MANIPULAR UN PUNTERO CON MÁS DE UN ÍNDICE

NO PUEDE EMPLEAR ARREGLOS AUXILIARES, ESTÁTICOS O DINÁMICOS, PARA GUARDAR LOS DATOS DEL ARCHIVO

LOS ARCHIVOS SOLO SE PUEDEN LEER UNA VEZ

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 2 del curso: "Arreglos y punteros". En este laboratorio se trabajará con memoria dinámica, punteros genéricos y punteros a función.

El método de ordenamiento **Bubble Sort** (ordenamiento burbuja) organiza los elementos de un arreglo mediante comparaciones entre elementos consecutivos.

El algoritmo recorre el arreglo de datos comparando cada par de elementos consecutivos (elementos $[i]$ e $[i+1]$). Si ambos elementos están en desorden, se intercambian de posición.

Por ejemplo, si queremos ordenar un arreglo de forma ascendente y tenemos los valores: [5, 2, 8, 1].

Primero se comparan 5 y 2. Como $5 > 2$, se intercambian: [2, 5, 8, 1].

Luego se comparan 5 y 8. Como ya están en el orden correcto, no se realiza ningún cambio.

Después se comparan 8 y 1, por lo que se intercambian: [2, 5, 1, 8]

Al terminar el recorrido, el valor más alto queda al final del arreglo, como una burbuja que sube hasta la superficie. Luego se repite el proceso, pero esta vez no se considera el último elemento porque este elemento ya se ubicó en su posición correcta. El proceso se repite hasta que todos los elementos queden en su posición correcta y el arreglo se encuentre totalmente ordenado.

"Bubble Sort - Mejorado": El algoritmo anterior tiene una eficiencia de $O(n^2)$ por lo que no se considera eficiente. Sin embargo, este proceso se puede mejorar mucho de manera muy sencilla agregando estos dos cambios: si al hacer un recorrido en el arreglo se detecta que no se produjeron intercambios entonces el arreglo ya se habrá ordenado y se da por terminada la ordenación y si durante un recorrido se determina que el último intercambio se produjo en la posición k del arreglo, el siguiente recorrido solo debe llegar a esa posición ya que el resto de los elementos estarán en su posición ordenada.

Por ejemplo, si queremos ordenar un arreglo de forma ascendente y tenemos los valores:

0	1	2	3	4	5	6	7
8	5	3	15	1	17	21	33

El primer recorrido llegará hasta el elemento 6 ya que no se tiene idea de cómo están los datos. Sin embargo, luego del primer recorrido el arreglo quedaría de la siguiente manera:

0	1	2	3	4	5	6	7
5	3	8	1	15	17	21	33

El último intercambio se habrá realizado en la posición 3, donde se intercambiaron el 15 con el 1, por esa razón el siguiente recorrido ya no llegará hasta el elemento 5 sino hasta el elemento 3 donde se produjo el último intercambio. Si no se hubiera producido algún intercambio, el proceso deba parar.

Deberá elaborar un proyecto denominado "[PunterosAFuncion_2026_2](#)" y en él desarrollará el programa que dé solución a las preguntas 1 y 2 planteadas. [DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL.](#)

Pregunta 1 (4 puntos)

Defina una biblioteca denominada "**FuncionesDeOrdenacion**" (.h/.hpp y .cpp) y allí defina una función denominada "**bubbleSort**" que implemente el método de ordenación "**Bubble Sort - Mejorado**", tal como se explica en los dos párrafos anteriores, para ordenar valores enteros contenidos en un arreglo. La función debe devolver el número de recorridos que se hicieron al arreglo para ordenarlo. La función no lee ni imprime datos, recibe el arreglo con los datos y el número de elementos que tiene. En la función main debe probar la función con un arreglo con los siguientes valores:

17,11,3,19,12,5,22,32,10,7,29,31,47,26,2,45,18,37,41,38 (son 20 datos)

Al probar la función debe obtener que el número de recorridos debe salir 14 (± 1).

Pregunta 2 (6 puntos)

Esta parte depende de la correcta implementación de la Parte 1.

Implemente en la misma biblioteca una función denominada **"bubbleSortGenerica"** que, modificando el código que empleó en la pregunta 1, pueda ordenar cualquier arreglo de punteros (int **, char **, double **, void *, int ***, char ***, etc.). Pruebe esta función. Para esto último implemente otra biblioteca denominada **"funcionesPruebas"** donde colocará todas las funciones que crea conveniente para poder leer los datos de los archivos "Numeros.txt" y "Nombres.txt", ordenarlos y mostrar cada uno en un reporte. Indicar el número de veces que se recorre en cada caso. Debe emplear métodos eficientes para almacenar los datos.

Pregunta 3 (10 puntos)

Esta parte depende de la correcta implementación de la Parte 2.

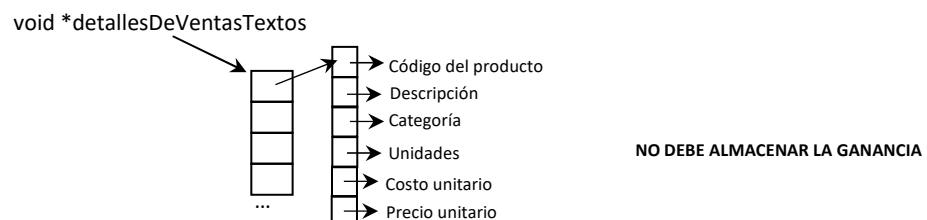
Deberá elaborar un proyecto denominado **"PunterosAFuncion_Ordenación_2026_2"** y en él desarrollará el programa que dé solución a la siguiente pregunta. **DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL.**

PackMart es una empresa ficticia dedicada a la venta minorista y mayorista de productos de consumo. La empresa administra productos que pueden venderse por unidad base o mediante presentaciones comerciales, por ejemplo, paquetes de 10, 20, 25 o 50 unidades y cajas de 12, 24, 48 o 100 unidades. Entre otros, la empresa maneja el siguiente archivo.

Productos.csv						
P0001,	Avena Pacifico Tipo 17,	ABARROTES,	153,	22,	25.35	
P0002,	Bebida Isotonica Costa Tipo 24,	BEBIDAS,	225,	40.50,	56	
...						

Cada línea del archivo representa un producto comercializado por la empresa y contiene, en el siguiente orden: código del producto, descripción, categoría, unidades en almacén, costo y precio unitarios de venta.

Se pide que el programa lea los datos del archivo los coloque en una estructura como la que se muestra a continuación (no puede usar tipos de datos struct):



A continuación deberá emitir un reporte que permita verificar la correcta elaboración de la estructura. El reporte se denominará **ReporteDeProductosInicial.txt** como se muestra a continuación.

REPORTE DE CATÁLOGO DE PRODUCTOS						
Código	Descripcion del producto	Tipo	Unidades	Costo	Precio	Ganancia
P0001	Avena Pacifico Tipo 17	ABARROTES	153	22.00	25.35	512.55
P0002	Bebida Isotonica Costa Tipo 24	BEBIDAS	225	40.50	56.00	3487.50
...	...	...	...	...	...	...
Ganancia total: 45867.07						

Luego deberá ordenar los datos empleando la función **"bubbleSortGenerica"**, la ordenación deberá realizarse **ascendentemente por el tipo de producto y en caso de igualdad descendentemente por la ganancia obtenida**.

Finalmente, empleando la misma función que empleó en el reporte anterior genere el reporte ordenado con el nombre **ReporteOrdenadoDeProductosInicial.txt**.

Profesores del curso: Erasmo Gómez Rony Cueva
 Ana Roncal Miguel Guanira

San Miguel, 2 de octubre del 2026.